



ПРОФЕСИОНАЛНА ГИМНАЗИЯ ПО КОМПЮТЪРНИ НАУКИ И  
МАТЕМАТИЧЕСКИ АНАЛИЗИ „ПРОФ. МИНКО БАЛКАНСКИ“

# ДИПЛОМЕН ПРОЕКТ

На тема:

**IoT Мониторинг**

Ученик: Никола Бориславов Панайотов, 12<sup>в</sup> клас

Професия: код 481 „Компютърни науки“

Специалност: код 4810201 „Системно програмиране“

<https://github.com/N1ki2K/Iot-Monitoring>

Ръководител консултант: Христо Стоев

град Стара Загора

2026 година



## СЪДЪРЖАНИЕ

|                                                                    |    |
|--------------------------------------------------------------------|----|
| 1. УВОД.....                                                       | 3  |
| 1.1 Контекст и значимост на темата .....                           | 3  |
| 1.2 Формулиране на проблема .....                                  | 3  |
| 1.3 Цел на дипломния проект .....                                  | 4  |
| 1.4 Очаквани резултати.....                                        | 4  |
| 2. АНАЛИЗ И СПЕЦИФИКАЦИЯ НА ИЗИСКВАНИЯТА.....                      | 5  |
| 2.1 Анализ на предметната област .....                             | 5  |
| 2.2 Сравнение на технологиите по слой.....                         | 5  |
| 2.3 Потребителски роли .....                                       | 6  |
| 2.4 Функционални изисквания .....                                  | 6  |
| 2.5 Нефункционални изисквания .....                                | 7  |
| 2.5.1 Сигурност .....                                              | 7  |
| 2.5.2 Производителност .....                                       | 7  |
| 2.5.3 Надеждност .....                                             | 7  |
| 2.5.4 Поддържаемост.....                                           | 7  |
| 3. АРХИТЕКТУРА НА СИСТЕМАТА И ПРОЕКТИРАНЕ НА БАЗАТА ДАННИ .....    | 8  |
| 3.1 Многослойна архитектура .....                                  | 8  |
| 3.2 Поток на данните.....                                          | 8  |
| 3.3 JWT автентикация и управление на сесиите.....                  | 9  |
| 3.4 Диаграма на последователността — Вход на потребител.....       | 10 |
| 3.5 Диаграма на последователността — Приемане на телеметрия.....   | 10 |
| 3.6 Диаграма на последователността — Заявяване на устройство ..... | 10 |
| 3.7 Проектиране на базата данни .....                              | 11 |
| 3.7.1 Концептуален ER модел .....                                  | 11 |
| 3.7.2 Таблица readings .....                                       | 11 |
| 3.7.3 Таблица users.....                                           | 12 |
| 3.7.4 Таблица refresh_tokens .....                                 | 12 |
| 3.8 Миграционна система .....                                      | 13 |
| 4. РЕАЛИЗАЦИЯ НА ПРОГРАМНОТО РЕШЕНИЕ .....                         | 14 |
| 4.1 Хардуерен модул — сензори и ESP32.....                         | 14 |
| 4.1.1 AM2302 (DHT22) — температура и влажност .....                | 14 |
| 4.1.2 ТЕМТ6000X01 — осветеност.....                                | 15 |
| 4.1.3 MQ-135 — качество на въздуха .....                           | 17 |
| 4.1.4 MAX4466 — ниво на звука.....                                 | 18 |



|                                                                 |    |
|-----------------------------------------------------------------|----|
| 4.2 Програмиране на ESP32.....                                  | 19 |
| 4.3 Модул за приемане на телеметрия.....                        | 20 |
| 4.4 API сървър.....                                             | 21 |
| 4.5 Логика за търсене и филтриране.....                         | 24 |
| 4.6 Уеб приложение с React.....                                 | 24 |
| 4.7 Мобилно приложение за Андроид.....                          | 25 |
| 4.8 Споделени TypeScript типове.....                            | 26 |
| 4.9 Технологичен стек.....                                      | 27 |
| 5. СИГУРНОСТ И НАДЕЖДНОСТ .....                                 | 28 |
| 5.1 Защита на паролите .....                                    | 28 |
| 5.2 JWT автентикация .....                                      | 28 |
| 5.3 Ролово разграничение на достъпа.....                        | 29 |
| 5.4 Одит лог.....                                               | 29 |
| 5.5 Текущи ограничения .....                                    | 29 |
| 5.6 Надеждност .....                                            | 29 |
| 6. ТЕСТВАНЕ И ВАЛИДИРАНЕ .....                                  | 30 |
| 6.1 Стратегия за тестване.....                                  | 30 |
| 6.2 Тестове на API сървъра.....                                 | 30 |
| 6.3 Тестове на уеб клиента .....                                | 31 |
| 6.4 Тестове на Андроид приложението.....                        | 31 |
| 6.5 Ръчна проверка .....                                        | 31 |
| 6.6 Обобщение .....                                             | 32 |
| 7. ЗАКЛЮЧЕНИЕ.....                                              | 33 |
| 7.1 Постигнати резултати .....                                  | 33 |
| 7.2 Оценка на изпълнението .....                                | 34 |
| 7.3 Предложения за бъдещо развитие .....                        | 34 |
| 8. ИЗПОЛЗВАНА ЛИТЕРАТУРА .....                                  | 36 |
| 9. ПРИЛОЖЕНИЯ .....                                             | 38 |
| Приложение А — Технически листове на сензорите .....            | 38 |
| Приложение Б — Environment Variables .....                      | 38 |
| Приложение В — Качествени характеристики .....                  | 39 |
| Приложение Г — Екранни снимки на потребителския интерфейс ..... | 39 |



## **1. УВОД**

### **1.1 Контекст и значимост на темата**

Светът е в разгара на четвъртата индустриална революция, при която физическите и цифровите пространства се сливат по безпрецедентен начин. В основата на тази промяна стои Интернетът на нещата — технологична парадигма, при която физически предмети се снабдяват с изчислителна мощ, сензори и мрежова свързаност, позволяваща им да събират, обработват и обменят данни без пряка намеса от страна на човека. Към 2025 г. в света функционират над 17 милиарда свързани устройства, а прогнозите предвиждат удвояване на тази бройка до 2030 г.

Наблюдението на параметрите на средата — температура, влажност, качество на въздуха, осветеност и ниво на шум — е сред най-разпространените практически приложения в жилищни, образователни и офисни пространства. Изследвания показват, че повишената концентрация на въглероден диоксид в затворени помещения влошава концентрацията и резултатите на учениците, а недостатъчното или прекалено силно осветление водят до умора. Точно такива проблеми могат да бъдат проследявани и решавани с подобна наблюдателна система.

Освен конкретния приложен ефект, изграждането на системата обхваща ключови понятия от учебната програма по системно програмиране: вградени системи и управление на хардуер, мрежови протоколи и обмен на данни, релационни бази данни, уеб разработка от страна на сървъра и на потребителя, мобилна разработка за операционната система Андроид, и сигурност на приложенията.

### **1.2 Формулиране на проблема**

В типична образователна или домашна среда суровите стойности от сензорите са практически неизползваеми без няколко условия. Данните трябва да се събират непрекъснато и автоматично. Необходимо е те да се съхраняват исторически за ретроспективен анализ на тенденциите. Достъпът до тях трябва да е възможен дистанционно от различни устройства. Данните трябва да са свързани с конкретни потребители чрез система за права. Накрая, те трябва да са представени в разбираем визуален вид.



Без централна наблюдателна платформа показанията остават изолирани на хардуерното устройство, недостъпни за анализ с течение на времето и неизползваеми от множество потребители едновременно. Настоящият проект адресира всеки един от тези проблеми, изграждайки пълен технологичен конвейер от физическото сензорно измерване до потребителски табла в уеб и мобилни приложения, с поддръжка на ролово разграничение на достъпа и пълна проследяемост на действията.

### **1.3 Цел на дипломния проект**

Основната цел е да се проектира и реализира пълноценна платформа за наблюдение на данни от Интернет на нещата, която придобива измервания от сензорно оборудвани контролери на базата на ESP32 и ги доставя надеждно и сигурно до потребителите чрез уеб и мобилни приложения. Проектът демонстрира прилагане на принципите на съвременното софтуерно инженерство при изграждането на реална многослойна система с пълен стек от технологии, съответстваща на изискванията за техническа документация.

### **1.4 Очаквани резултати**

- Функционираща платформа за наблюдение, свързваща ESP32 контролер с уеб и мобилни приложения
- REST API с JWT автентикация, покриващ пълния набор от функционални изисквания
- Уеб табло за управление с показания в близко до реално време и исторически графики
- Приложение за Андроид с компонент на началния екран на устройството
- PostgreSQL база данни с пълна история на измерванията и управление на сесиите
- Ролова система за управление на достъпа с одит лог
- Документирана многослойна архитектура с реална сензорна хардуерна интеграция



## 2. АНАЛИЗ И СПЕЦИФИКАЦИЯ НА ИЗИСКВАНИЯТА

### 2.1 Анализ на предметната област

Системите за наблюдение на параметрите на средата се характеризират с три основни компонента, всеки от които изпълнява ясно разграничена роля. Физическият слой включва сензори и микроконтролери, отговорни за действителното измерване на физически величини. Комуникационният слой осигурява пренос на данните от устройствата до сървъра. Приложният слой обхваща сървъри, бази данни и потребителски интерфейси за съхранение, обработка и визуализация.

При проектирането е направен анализ на водещите съществуващи платформи. Grafana с InfluxDB предлага богата визуализация, но изисква значителна инфраструктурна конфигурация и не дава дълбоко разбиране на вътрешната работа. ThingSpeak предоставя облачно съхранение, но е с ограничена гъвкавост при потребителски роли. AWS IoT Core и Azure IoT Hub са промишлени решения с абонаментни разходи и висока сложност. Настоящият проект избира собствена реализация — напълно контролируема, безплатна, локално разгъваема и образователно ценна.

### 2.2 Сравнение на технологиите по слой

| Слой                    | Разгледана алтернатива | Избор                | Обосновка                                                                             |
|-------------------------|------------------------|----------------------|---------------------------------------------------------------------------------------|
| Съобщителен протокол    | HTTP опросване         | MQTT                 | По-малко натоварване; модел издател-абонат; устройството не зависи от сървъра         |
| MQTT посредник          | HiveMQ, EMQ X          | Eclipse Mosquitto    | Лек, с отворен код, лесен за локална инсталация                                       |
| База данни              | InfluxDB, MongoDB      | PostgreSQL           | Трайност и последователност; релационен модел за телеметрия и бизнес обекти           |
| Обработка на сървъра    | Python/Django, Go      | Node.js + TypeScript | Управлявано от събития входно-изводно; статично типизиране; единен стек с уеб клиента |
| Потребителски интерфейс | Vue, Angular           | React 19             | Компонентна архитектура; hooks за управление на състоянието; голяма общност           |



|                    |                       |                   |                                                                                        |
|--------------------|-----------------------|-------------------|----------------------------------------------------------------------------------------|
| Мобилна разработка | Flutter, React Native | Kotlin за Андроид | Стандартен Андроид стек; компонент на началния екран; безопасност при нулеви стойности |
| Контейнеризация    | Kubernetes            | Docker Compose    | Лесно повторяемо локално разгъване; минимална сложност                                 |

## 2.3 Потребителски роли

| Роля                     | Описание               | Права за достъп                                                                                                                                  |
|--------------------------|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Обикновен потребител     | Краен потребител       | Управление на собствения профил; преглед само на назначените устройства; заявяване на устройства с код за вдвояване; достъп до исторически данни |
| Администратор (is_admin) | Системен администратор | Пълен достъп до всички потребители, контролери и данни; управление на роли; покана на потребители; преглед на одит лога и системното здраве      |

## 2.4 Функционални изисквания

| Код   | Функционално изискване                                                            | Статус |
|-------|-----------------------------------------------------------------------------------|--------|
| ФИ-01 | Регистрация и вход с потребителско име, парола и електронна поща.                 | ✓      |
| ФИ-02 | Разграничаване на потребителски роли — обикновен потребител и администратор.      | ✓      |
| ФИ-03 | Пълен достъп на администратор до всички данни и функционалности.                  | ✓      |
| ФИ-04 | Събиране на данни от устройство на Интернет на нещата (ESP32 с пет типа сензори). | ✓      |
| ФИ-05 | Автоматично изпращане на данните към сървъра чрез протокола MQTT.                 | ✓      |
| ФИ-06 | Трайно съхранение на данните в PostgreSQL база данни.                             | ✓      |
| ФИ-07 | Визуализация в уеб и мобилен интерфейс под формата на таблици и графики.          | ✓      |
| ФИ-08 | Обновяване на информацията в близко до реалното време.                            | ✓      |
| ФИ-09 | Съхраняване и преглед на история с дата и час.                                    | ✓      |
| ФИ-10 | Обратна връзка при грешки или липсващи данни.                                     | ✓      |



В допълнение са реализирани: JWT автентикация с refresh token ротация; заявяване на контролери с петцифрен код за вдвояване; управление на потребителско-контролерни отношения; одит лог на чувствителните операции; системни показатели за администраторите.

## **2.5 Нефункционални изисквания**

### **2.5.1 Сигурност**

Паролите се съхраняват само като хешове, генерирани с алгоритъма `bcrypt` — изчислително скъпа функция с висока памет, устойчива на GPU-базирани атаки. Достъпът до API се защитава чрез подписани JWT access токени с кратък живот. Refresh tokenите се съхраняват като SHA-256 хешове в базата данни и подлежат на ротация и отмяна. Всеки потребител вижда само устройствата, назначени му лично.

### **2.5.2 Производителност**

Системата трябва да обработва постъпващи измервания с закъснение под 1 секунда. Таблото за управление се обновява автоматично на всеки 30 секунди. Таблицата с измервания е индексирана по двойката (идентификатор на устройството, времеви печат в намаляващ ред) за бързи заявки.

### **2.5.3 Надеждност**

PostgreSQL осигурява трайност и последователност на записаните данни. Програмирането на ESP32 включва логика за автоматично повторно свързване при загуба на мрежата. Разделянето на слоя за приемане от API сървъра означава, че измерванията продължават да се записват дори при временен срив на API.

### **2.5.4 Поддържамост**

Монорепо структурата с работни пространства на прт осигурява единна точка за управление на зависимостите. Споделеният пакет с TypeScript типове гарантира последователност на интерфейсите между сървъра и уеб клиента. Промените в схемата се управляват чрез последователни SQL миграции.

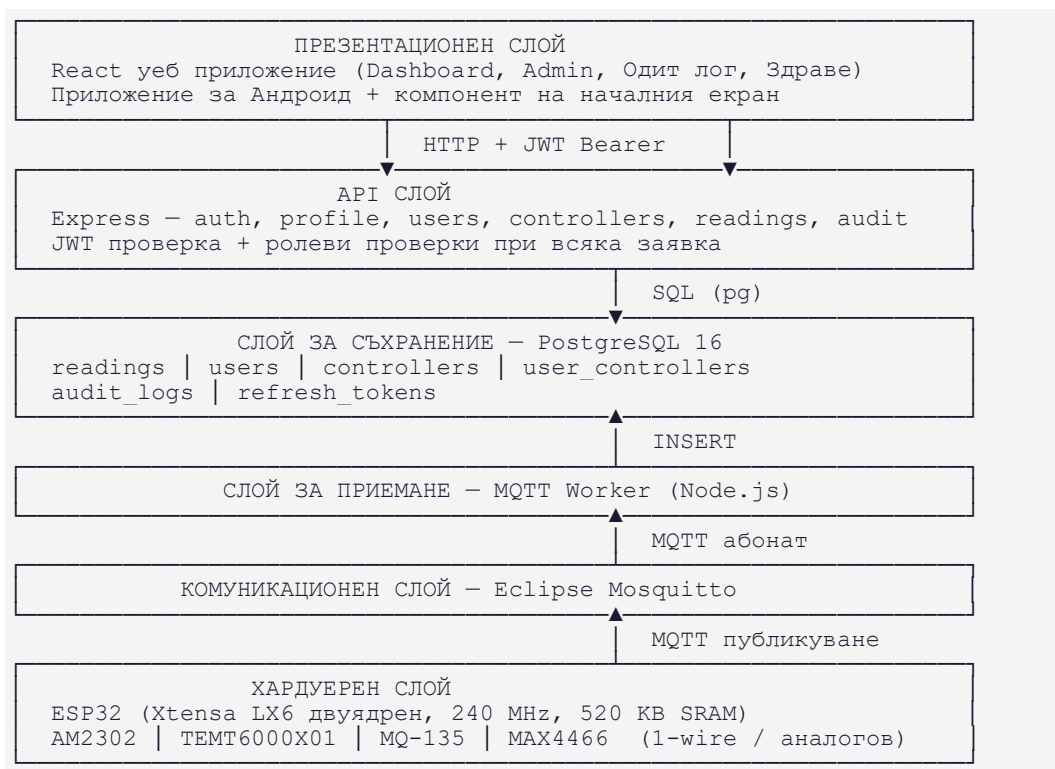




### 3. АРХИТЕКТУРА НА СИСТЕМАТА И ПРОЕКТИРАНЕ НА БАЗАТА ДАННИ

#### 3.1 Многослойна архитектура

Системата следва строго разслоена архитектура, при която всеки слой има ясно определена отговорност и общува само с непосредствено съседните слоеве. Тази организация произтича от добре установения принцип на разделяне на отговорностите — всяка съставна част се занимава само с проблемите, за чието решаване е проектирана. Промяна в един слой не налага промени в другите, ако начинът им на обмен на данни остава непроменен.



#### 3.2 Поток на данните

Разбирането на пълния поток на данните показва, че системата е пълноценен конвейер с придобиване, пренос, проверка, съхранение, оторизация и представяне — не просто четец на сензори.

1. Физическите сензори измерват параметрите на средата
2. ESP32 чете стойностите на всеки 5 секунди
3. Програмирането пакетира данните в JSON:  
{ "t":24.5, "h":61.2, "lux":1200, "sound":430, "aq":780 }



```
4. ESP32 публикува JSON към MQTT посредника (тема: iot/esp32/telemetry)
5. Mosquitto препраща съобщението към абонираните слушатели
6. Модулът за приемане на телеметрия извлича device_id от темата,
   анализира JSON
7. INSERT INTO readings (device_id, temperature_c, ...)
8. Клиент изпраща: GET /api/latest/esp32
   Authorization: Bearer <access_token>
9. JWT middleware проверява подписа и валидността на токена
10. Проверка на device access права чрез user_controllers
11. API връща JSON отговор
12. Клиентът рендерира данните като карти, графики и таблици
```

### 3.3 JWT автентикация и управление на сесиите

Системата използва JWT (JSON Web Token) автентикация с refresh token ротация. Тази схема е значително по-сигурна от по-ранния подход с обикновен хедър и представлява практически приложим модел за курсова работа и малки разгъвания.

При успешен вход сървърът издава два токена: кратко живеещ access token (JWT, подписан с таен ключ) и дългоживеещ opaque refresh token. Access tokenът се изпраща при всяка защитена заявка в Authorization хедъра. Когато access tokenът изтече, клиентът може да поиска нов чрез refresh токена. Refresh tokenът не се съхранява в оригинален вид — сървърът записва само неговия SHA-256 хеш в таблицата refresh\_tokens заедно с датата на изтичане и евентуална дата на отмяна.

```
// Поток за автентикация:

POST /api/auth/login { email, password }
→ { token: 'eyJ...', refreshToken: 'abc123...' }

// Защитена заявка:
GET /api/me
  Authorization: Bearer eyJ...
  → { id, username, email, role, is_admin }

// Обновяване на access токена:
POST /api/auth/refresh { refreshToken: 'abc123...' }
→ { token: 'eyJ...нов...', refreshToken: 'xyz789...' }
   // Старият refresh token се отбелязва като използван (ротация)

// Изход:
POST /api/auth/logout { refreshToken: 'xyz789...' }
→ Refresh tokenът се отбелязва като отменен в базата данни
```

Ротацията на refresh tokenите означава, че при всяко обновяване старият token се обезсилва и се издава нов. Ако някой се опита да използва вече ротиран refresh token, сървърът ще отхвърли заявката като невалидна. Логиката за отмяна при изход позволява потребителят да прекрати сесията си по контролиран начин.

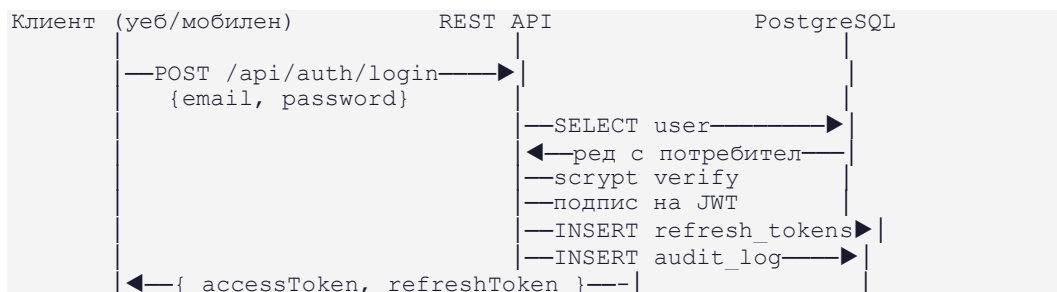
Животът на access токена и refresh токена се управлява чрез настройки на сървъра. В текущата реализация по подразбиране access tokenът е 3600 секунди, а



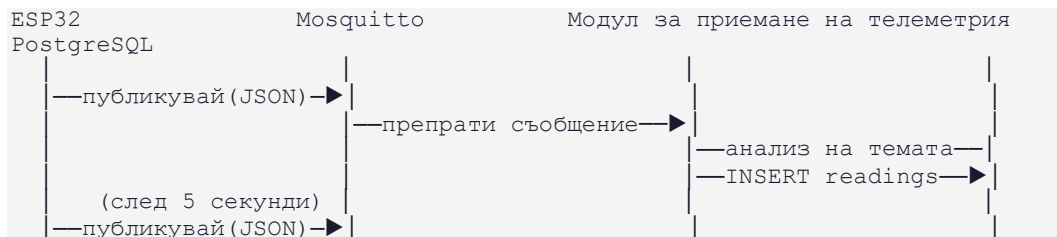
refresh токенът — 1209600 секунди (14 дни). Това ограничава прозореца за злоупотреба при евентуална кражба на access токен и позволява по-дълга, но контролируема потребителска сесия.

Полето revoked\_at в таблицата refresh\_tokens позволява ефективна проверка при всяко обновяване: сървърът търси реда по SHA-256 хеша и проверява дали revoked\_at е NULL и expires\_at е в бъдещето. Ако токенът е отменен или изтекъл, заявката за обновяване се отхвърля.

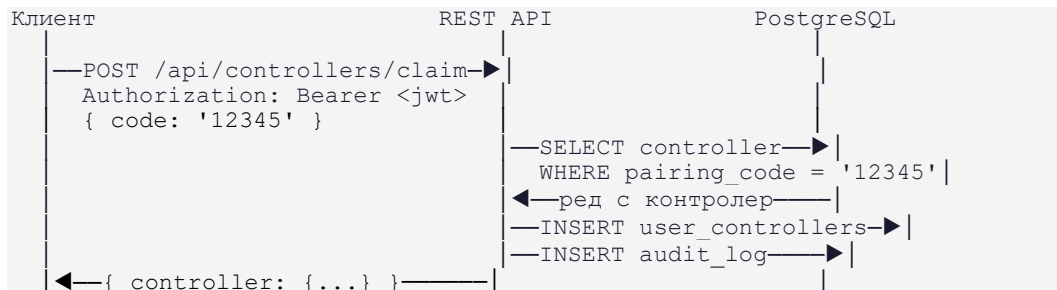
### 3.4 Диаграма на последователността — Вход на потребител



### 3.5 Диаграма на последователността — Приемане на телеметрия



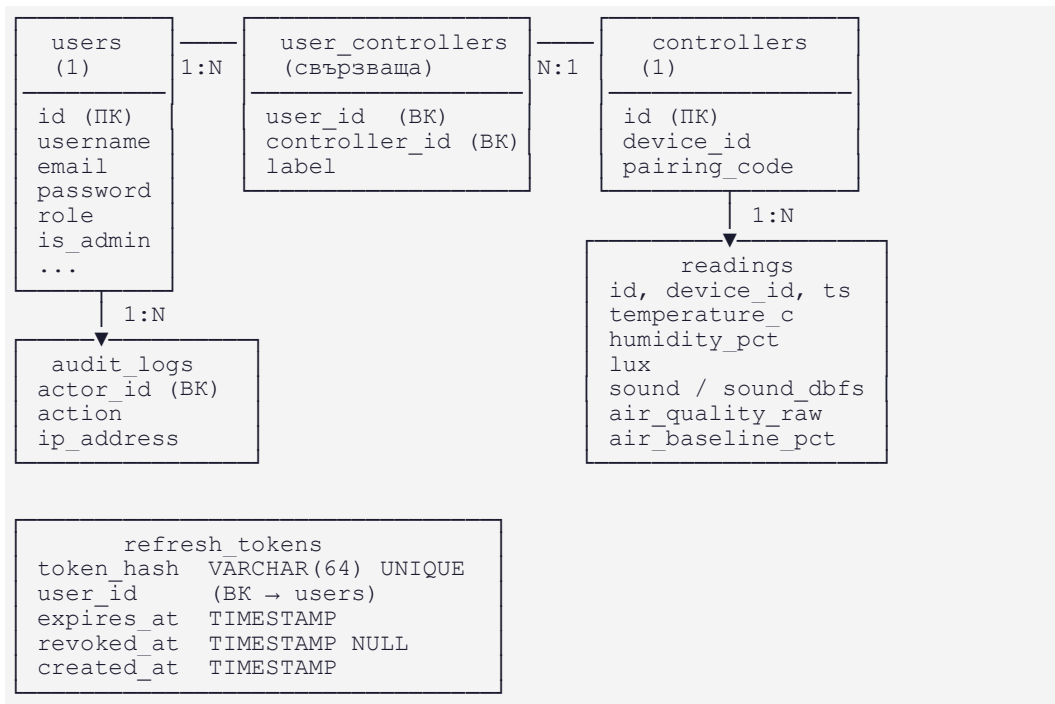
### 3.6 Диаграма на последователността — Заявяване на устройство





### 3.7 Проектиране на базата данни

#### 3.7.1 Концептуален ER модел



#### 3.7.2 Таблица readings

| Колона        | Тип данни               | Описание                                            |
|---------------|-------------------------|-----------------------------------------------------|
| id            | SERIAL PRIMARY KEY      | Уникален пореден номер на записа                    |
| device_id     | VARCHAR(64) NOT NULL    | Идентификатор на ESP32 устройството                 |
| ts            | TIMESTAMP DEFAULT NOW() | Точен времеви печат на измерването                  |
| temperature_c | DECIMAL(5,2)            | Температура в °C (от AM2302)                        |
| humidity_pct  | DECIMAL(5,2)            | Относителна влажност в % (от AM2302)                |
| lux           | INTEGER                 | Осветеност — сурова стойност от TEMT6000X01         |
| sound         | INTEGER                 | Ниво на звука — сурова стойност от MAX4466 (0–4095) |
| sound_dbfs    | DECIMAL(6,2)            | Ниво на звука в dBFS (изчислена)                    |



|                  |              |                                                 |
|------------------|--------------|-------------------------------------------------|
| sound_est_spl    | DECIMAL(6,2) | Приблизително ниво на звуковото налягане в dB   |
| air_quality_raw  | INTEGER      | Качество на въздуха — сурова стойност от MQ-135 |
| air_baseline_pct | DECIMAL(5,2) | Качество на въздуха в % спрямо базова стойност  |

### 3.7.3 Таблица users

| Колона               | Тип данни                       | Описание                            |
|----------------------|---------------------------------|-------------------------------------|
| id                   | SERIAL PRIMARY KEY              | Уникален пореден номер              |
| username             | VARCHAR(64)<br>UNIQUE NOT NULL  | Потребителско име                   |
| email                | VARCHAR(255)<br>UNIQUE NOT NULL | Адрес на електронна поща            |
| password             | TEXT NOT NULL                   | сcrypt хеш на паролата              |
| role                 | TEXT DEFAULT<br>'user'          | Роля: user / admin                  |
| is_admin             | BOOLEAN<br>DEFAULT FALSE        | Флаг за административни права       |
| invited_by           | INTEGER → users.id              | Кой е поканил потребителя (BK)      |
| must_change_password | BOOLEAN<br>DEFAULT FALSE        | Принудителна смяна при следващ вход |
| created_at           | TIMESTAMP<br>DEFAULT NOW()      | Дата и час на регистрация           |

### 3.7.4 Таблица refresh\_tokens

Съхранява активните и отменените сесии за refresh токени. Всеки ред отговаря на една издадена refresh сесия. Токенът не се пази в оригинален вид — само SHA-256 хешът му се записва, което означава, че дори пробив на базата данни не позволява извличане на валидни токени.

| Колона     | Тип данни                      | Описание                      |
|------------|--------------------------------|-------------------------------|
| id         | SERIAL PRIMARY KEY             | Уникален пореден номер        |
| token_hash | VARCHAR(64)<br>UNIQUE NOT NULL | SHA-256 хеш на refresh токена |



|            |                         |                                   |
|------------|-------------------------|-----------------------------------|
| user_id    | INTEGER → users.id      | За кой потребител е сесията (ВК)  |
| expires_at | TIMESTAMP NOT NULL      | Дата и час на изтичане на сесията |
| revoked_at | TIMESTAMP NULL          | Дата на отмяна (NULL = активна)   |
| created_at | TIMESTAMP DEFAULT NOW() | Дата и час на създаване           |

### 3.8 Миграционна система

| Миграция                       | Промяна в схемата                                          |
|--------------------------------|------------------------------------------------------------|
| 001_create_audit_logs.sql      | Създаване на таблица audit_logs                            |
| 002_add_user_role.sql          | Добавяне на колона role в таблица users                    |
| 003_add_user_is_admin.sql      | Добавяне на булев флаг is_admin                            |
| 004_add_user_invite_fields.sql | Полета invited_by, invited_at, must_change_password        |
| 005_add_sound_dbfs.sql         | Добавяне на sound_dbfs в таблицата с измервания            |
| 006_add_sound_est_spl.sql      | Добавяне на sound_est_spl в таблицата с измервания         |
| 007_add_air_baseline_pct.sql   | Добавяне на air_baseline_pct в таблицата с измервания      |
| 008_rename_co2_ppm.sql         | Преименуване на co2_ppm в air_quality_raw                  |
| 009_drop_legacy_user_flag.sql  | Премахване на стари флагове на потребители                 |
| 010_create_refresh_tokens.sql  | Създаване на таблица refresh_tokens за управление на сесии |



## 4. РЕАЛИЗАЦИЯ НА ПРОГРАМНОТО РЕШЕНИЕ

### 4.1 Хардуерен модул — сензори и ESP32

Хардуерният слой на системата се основава на микроконтролера ESP32 с четири прикачени сензора, всеки отговарящ за измерването на различен физически параметър на средата. Изборът на всеки сензор е направен на база на достъпност, точност и съвместимост с ESP32.

#### 4.1.1 AM2302 (DHT22) — температура и влажност

Сензорът AM2302 е цифров модул за измерване на температура и относителна влажност с изградена вградена калибрация. Произведен от Aosong Electronics, той използва капацитивен влагочувствителен елемент и прецизен термометър, свързани с 8-битов микроконтролер, отговарящ за обработката и предаването на данните.

Комуникацията с ESP32 се осъществява по протокол с единична жица (1-wire / single-bus). При всяко запитване сензорът предава 40 бита данни: 16 бита влажност, 16 бита температура и 8 бита контролна сума за проверка. Данните са с разрешителна способност от 0,1 °C и 0,1% RH, а стойностите са 10 пъти завишени спрямо действителните за избягване на десетичен запетаен знак при предаването.

| Параметър                              | Минимум | Типично | Максимум | Единица |
|----------------------------------------|---------|---------|----------|---------|
| Работно напрежение                     | 3.3     | 5.0     | 5.5      | V       |
| Обхват на температура                  | -40     | —       | 80       | °C      |
| Точност на температура                 | —       | ±0.5    | ±1       | °C      |
| Разрешителна способност на температура | —       | 0.1     | —        | °C      |
| Обхват на влажност                     | 0       | —       | 99.9     | %RH     |
| Точност на влажност (при 25°C)         | —       | —       | ±2       | %RH     |
| Разрешителна способност на влажност    | —       | 0.1     | —        | %RH     |
| Скорост на реакция (влажност)          | —       | <5      | —        | s       |



|                                     |   |      |   |          |
|-------------------------------------|---|------|---|----------|
| Дрейф на влажност                   | — | <0.5 | — | %RH/год. |
| Минимален интервал между измервания | 2 | —    | — | s        |
| Максимална дължина на кабел         | — | 20   | — | m        |

Важна особеност при работа с AM2302 е минималният интервал от 2 секунди между последователни измервания. Прочетената стойност винаги е от последното измерване — за получаване на актуални данни е препоръчително непрекъснато циклично четене с интервал над 2 секунди. В проекта е използван интервал от 5 секунди, което е в рамките на изискванията и намалява натоварването на сензора.

Протоколът с единична жица на AM2302 предава данните като 40-битова серийна последователност: 16 бита за влажност (старши байт първи), 16 бита за температура (старши байт първи) и 8 бита контролна сума. Контролната сума е сумата от четирите байта данни по модул 256. Отрицателните температури се кодират с вдигнат най-висок бит на температурния байт. В Arduino програмирането библиотеката DHT.h управлява целия протокол автоматично, включително времевите изисквания за стартовия сигнал (поне 800  $\mu$ s ниско ниво) и обработката на контролната сума.

Протоколът с единична жица изисква pull-up резистор от около 5,1 k $\Omega$  между линията за данни и захранването. При захранване от 3,3 V дължината на кабела не трябва да надвишава 100 cm, тъй като падът на напрежение влиза в грешки при измерване. В проекта сензорът е захранван с 5 V и е свързан директно до ESP32 GPIO пина.

#### 4.1.2 ТЕМТ6000X01 — осветеност

Сензорът ТЕМТ6000X01 на Vishay Semiconductors е силициев NPN епитаксиален планарен фототранзистор, специално проектиран за измерване на видима светлина, наподобявайки реакцията на човешкото око. Той е в компактен SMD корпус 1206 (4  $\times$  2  $\times$  1,05 mm) и е подходящ за директен монтаж на печатна платка.

Сензорът е чувствителен към видимата светлина в диапазона 440–800 nm с пикова чувствителност при 570 nm — точно в жълто-зелената зона на спектъра, където чувствителността на човешкото око е максимална. Ъгълът на полуредукция на





чувствителността е  $\pm 60^\circ$ , което означава широко приемно поле и е удобно за измерване на общата осветеност в помещение без нужда от прецизно насочване.

| Параметър                           | Минимум | Типично  | Максимум | Единица          |
|-------------------------------------|---------|----------|----------|------------------|
| Работно напрежение (Vce)            | —       | 5        | 6        | V                |
| Фототок при 20 lx (Vce=5V)          | 3.5     | 10       | 16       | $\mu\text{A}$    |
| Фототок при 100 lx (Vce=5V)         | —       | 50       | —        | $\mu\text{A}$    |
| Пикова дължина на вълната           | —       | 570      | —        | nm               |
| Спектрален обхват (0.5)             | 440     | —        | 800      | nm               |
| Ъгъл на полуредукция                | —       | $\pm 60$ | —        | $^\circ$         |
| Температурен коефициент на фототока | —       | 0.9      | —        | %/K              |
| Максимален ток на колектора         | —       | —        | 20       | mA               |
| Работна температура                 | -40     | —        | +100     | $^\circ\text{C}$ |

В проекта TEMT6000X01 е свързан към аналоговия вход на ESP32. Фототокът се преобразува в напрежение чрез товарен резистор, а аналого-цифровият преобразувател (АЦП) на ESP32 го преобразува в 12-битова цифрова стойност (0–4095). Получената стойност се записва директно в базата данни като сурово число, тъй като при проект за сравнително наблюдение относителните промени са по-важни от абсолютните стойности в лукс.

Линейността на TEMT6000X01 е забележителна — фототокът нараства линейно с осветеността в диапазона 1–1000 лукс (съгласно графиката Photo Current vs. Illuminance от datasheet-a). Температурният коефициент на фототока при бяла LED е 0,9 %/K, което означава незначителна промяна при нормални стайни условия. Ъгловата характеристика показва, че сензорът запазва над 70% от пиковата си чувствителност до  $\pm 40^\circ$  от вертикалата и пада до около 50% при  $\pm 60^\circ$ . При монтиране на плоска повърхност сензорът улавя светлина от широк телесен ъгъл, което е желателно за измерване на общата осветеност в помещение.



#### 4.1.3 MQ-135 — качество на въздуха

Сензорът MQ-135 е полупроводников газов сензор, използващ SnO<sub>2</sub> (калаен диоксид) като чувствителен материал. SnO<sub>2</sub> има по-висока съпротива в чист въздух — с нарастването на концентрацията на замърсители съпротивата на сензора намалява. Тази промяна на съпротивата се измерва чрез товарен резистор и АЦП на ESP32.

MQ-135 е чувствителен към широк спектър от газове, присъстващи в замърсения въздух на затворени помещения: амоняк (NH<sub>3</sub>), азотни оксиди (NO<sub>x</sub>), алкохол, бензен, дим и въглероден диоксид (CO<sub>2</sub>). Именно тази широка чувствителност го прави подходящ за обща оценка на качеството на въздуха, макар и без прецизно разграничаване между отделните газове.

| Параметър                                          | Минимум  | Типично | Максимум | Единица |
|----------------------------------------------------|----------|---------|----------|---------|
| Работно напрежение (V <sub>c</sub> )               | —        | 5       | —        | V       |
| Напрежение на нагревателя (V <sub>h</sub> )        | 5.0 ±0.1 | —       | —        | V       |
| Консумация на нагревателя                          | —        | —       | <800     | mW      |
| Съпротива на нагревателя                           | —        | 33 ±5%  | —        | Ω       |
| Обхват на чувствителна съпротива (R <sub>s</sub> ) | 30       | —       | 200      | kΩ      |
| Коефициент на наклон на концентрация               | —        | —       | ≤0.65    | —       |
| Работна температура                                | −10      | —       | +45      | °C      |
| Работна влажност                                   | —        | —       | <95      | %RH     |
| Изходно напрежение (аналогово)                     | 0        | —       | 5        | V       |
| Предварително загряване                            | —        | 20–30   | —        | s       |

Важна техническа особеност на MQ-135 е необходимостта от предварително загряване (preheat) преди измерване. Активният материал трябва да достигне работна температура около 300 °C, което отнема 20–30 секунди след включване. В проекта



ESP32 изчаква подходящ интервал след стартиране преди да започне публикуването на данни.

Калибрацията на MQ-135 се извършва чрез определяне на базовата съпротива  $R_o$  в чист въздух при 20 °C и 65% влажност. В проекта базовата стойност се измерва при инициализация и последващите стойности се изразяват като процент от нея (`air_baseline_pct`). Ако стойността е под 100%, въздухът е по-чист от базовото измерване. Ако е над 100%, концентрацията на газове е нараснала. Тази относителна скала е практична за образователен проект, тъй като не изисква сложна криви на калибрация по газов тип.

За прецизно измерване на PPM на конкретен газ чрез MQ-135 е необходимо да се определи стойността  $R_o$  (съпротивата на сензора при 100 ppm  $NH_3$  в чист въздух при 20°C и 65% влажност) и след това да се използва формулата  $R_s/R_o = f(ppm)$  от характеристикните криви в datasheet-а. Тъй като проектът цели обща индикация на качеството на въздуха, а не прецизно химично измерване, относителният подход с baseline е напълно оправдан. Важно е да се отбележи, че сензорът е чувствителен към температура и влажност — при различни условия едно и също газово замърсяване ще даде различна стойност. Затова показанията на MQ-135 трябва да се интерпретират заедно с данните от AM2302 за пълна картина.

#### 4.1.4 MAX4466 — ниво на звука

MAX4466 е нискошумен операционен усилвател на Maxim Integrated, оптимизиран специално за предусилване на микрофони. Той е член на серията MAX4465–MAX4469 — ултра-компактни микромощни усилватели в корпуси SC70 и SOT23. MAX4466 е декомпенсирана версия с минимално стабилно усилване +5 V/V и честотна лента 600 kHz.

В проекта MAX4466 усилва слабия аналогов сигнал от електретен микрофонен капсул и го подава към АЦП на ESP32. Усилвателят работи от 2,4 V до 5,5 V, консумира само 24  $\mu A$  в нормален режим и се характеризира с отлично отхвърляне на захранващи смущения ( $PSRR = 112\text{ dB}$ ) и общомодово отхвърляне ( $CMRR = 126\text{ dB}$ ) — качества, критични за работа в зашумена от цифрови компоненти среда.



| Параметър                                | Минимум | Типично | Максимум         | Единица                      |
|------------------------------------------|---------|---------|------------------|------------------------------|
| Работно захранване                       | 2.4     | —       | 5.5              | V                            |
| Консумация на ток (на усилвател)         | —       | 24      | 48               | $\mu\text{A}$                |
| Честотна лента (GBWP)                    | —       | 600     | —                | kHz                          |
| Минимално стабилно усилване              | —       | +5      | —                | V/V                          |
| Отхвърляне на захранващи смущения (PSRR) | 80      | 112     | —                | dB                           |
| Общомодово отхвърляне (CMRR)             | 80      | 126     | —                | dB                           |
| Шум на входното напрежение               | —       | 80      | —                | $\text{nV}/\sqrt{\text{Hz}}$ |
| Изходна осцилация (Rail-to-Rail)         | —       | —       | 16 mV от VCC/GND | mV                           |
| Работна температура                      | -40     | —       | +85              | $^{\circ}\text{C}$           |

Получената от АЦП 12-битова стойност (0–4095) се записва директно в базата данни. Допълнително програмирането изчислява dBFS (децибели спрямо пълна скала) и приблизително ниво на звуковото налягане (SPL), като преобразува суровата АЦП стойност чрез логаритмична формула. Тези производни стойности дават по-интерпретируеми резултати за потребителите.

Специфичното за MAX4466 е декомпенсираната му конструкция — за разлика от MAX4465 (стабилен при единично усилване), MAX4466 изисква минимално усилване от +5 V/V, за да остане стабилен. Тото гарантира честотна лента от 600 kHz, което е повече от достатъчно за гласова честота и измерване на нива на шум. Отличното общомодово отхвърляне от 126 dB означава, че усилвателят ефективно потиска смущенията от захранването и цифровите компоненти на ESP32, намалявайки фалшивите показания при измерване.

## 4.2 Програмиране на ESP32

Програмирането е реализирано като Arduino sketch в единствения файл device/esp32/init/init.ino. При стартиране ESP32 се свързва към конфигурираната Wi-



Fi мрежа и MQTT посредника, настройва АЦП резолюцията и започва периодично публикуване на измервания. В текущата реализация базовата стойност за качеството на въздуха е фиксирана константа AIR\_BASELINE\_RAW, а не динамична начална калибрация след изчакване.

```
// device/esp32/init/init.ino — публикуване на сензорни данни
void publishSensorData() {
  float temperature = safeReadTemperature();
  float humidity = safeReadHumidity();
  int lightRaw = analogRead(LIGHT_PIN);
  int airRaw = analogRead(AIR_PIN);
  int soundRaw = 0;
  float soundDbFs = readSoundDbFs(soundRaw);
  float soundEstSpl = estimateSoundSpl(soundDbFs);
  float airBaselinePct = estimateAirBaselinePct(airRaw);
  String payload = "{";
  payload += "\"t\":" + String(temperature, 2) + ",";
  payload += "\"h\":" + String(humidity, 2) + ",";
  payload += "\"lux\":" + String(lightRaw) + ",";
  payload += "\"sound\":" + String(soundRaw) + ",";
  payload += "\"sound_dbfs\":" + String(soundDbFs, 2) + ",";
  payload += "\"sound_est_spl\":" + String(soundEstSpl, 2) + ",";
  payload += "\"aq\":" + String(airRaw) + ",";
  payload += "\"air_baseline_pct\":" + String(airBaselinePct, 2);
  payload += "}";
  mqttClient.publish(TOPIC_DATA, payload.c_str());
}
serializeJson(doc, buf);
```

Изборът на JSON като формат за полезния товар остава обоснован с четимост при диагностика, лесно парсване в Node.js слоя и добра съвместимост между вградения код, MQTT преноса и PostgreSQL-backed backend. В текущата реализация JSON низът се изгражда ръчно чрез String, а имената на полетата остават кратки (t, h, lux, sound, aq), за да се намали размерът на MQTT съобщението.

### 4.3 Модул за приемане на телеметрия

Модулят за приемане на телеметрия (backend/src/ingest.ts) представлява отделен процес на Node.js, който е абониран за MQTT темата, зададена чрез променливата MQTT\_TOPIC. По подразбиране тя може да бъде конкретна тема, например `iot/esp32/telemetry`, но при необходимост може да се конфигурира и шаблон с wildcard.

При получаване на съобщение модулът извлича `device_id` от пътя на темата, анализира получените JSON данни и записва нов ред в таблицата `readings`.



```
// backend/src/ingest.ts
client.on('message', async (topic, payload) => {
  const deviceId = topic.split('/')[1]; // 'iot/esp32/telemetry'→'esp32'
  const data = JSON.parse(payload.toString());

  await pool.query(
    `INSERT INTO readings
      (device_id, temperature_c, humidity_pct, lux,
       sound, sound_dbfs, sound_est_spl,
       air_quality_raw, air_baseline_pct)
     VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9)` ,
    [deviceId, data.t, data.h, data.lux,
     data.sound, data.sound_dbfs ?? null, data.sound_est_spl ?? null,
     data.aq, data.air_baseline_pct ?? null]
  );
});
```

## 4.4 API сървър

API сървърът (backend/src/api.ts) е изграден с Express.js и е организиран в шест модула с маршрути. JWT проверката се изпълнява чрез authContextMiddleware и помощни функции като getRequester и requireRequester, които валидират Bearer токена и при нужда зареждат потребителя от базата данни.

```
// backend/src/api/common.ts – опростен откъс от JWT логиката
export const authContextMiddleware = async (req, _res, next) => {
  const authHeader = req.header('authorization');
  const tokenMatch = authHeader?.match(/^(Bearer\s+(.+)$/i);
  if (!tokenMatch) { req.auth = { failureReason: 'missing' }; return
  next(); }
  try {
    req.auth = { tokenPayload: await
  verifyAccessToken(tokenMatch[1].trim()) };
  } catch {
    req.auth = { failureReason: 'invalid' };
  }
  next();
};

export const requireRequester = async (req, res) => {
  const requester = await getRequester(req);
  if (!requester) return res.status(401).json({ error: 'invalid bearer
  token' });
  return requester;
};
```

Пълен списък на реализираните маршрути:

| Метод | Адрес              | Описание                             |
|-------|--------------------|--------------------------------------|
| POST  | /api/auth/register | Регистрация на нов потребител        |
| POST  | /api/auth/login    | Вход — връща access и refresh токени |



|        |                                    |                                                  |
|--------|------------------------------------|--------------------------------------------------|
| POST   | /api/auth/refresh                  | Обновяване на access token с refresh token       |
| POST   | /api/auth/logout                   | Изход — отменя refresh токена                    |
| GET    | /api/me                            | Профил на текущия потребител                     |
| PATCH  | /api/me                            | Актуализиране на профилни данни                  |
| PATCH  | /api/me/password                   | Промяна на парола                                |
| DELETE | /api/me                            | Изтриване на собствения профил                   |
| GET    | /api/users                         | Списък на потребителите (администратор)          |
| GET    | /api/users/:userId                 | Профил на конкретен потребител (администратор)   |
| PATCH  | /api/users/:userId                 | Редакция на потребителски профил (администратор) |
| DELETE | /api/users/:userId                 | Изтриване на потребител (администратор)          |
| PATCH  | /api/users/:userId/role            | Промяна на роля (администратор)                  |
| POST   | /api/admin/users/invite            | Покана на нов потребител (администратор)         |
| GET    | /api/controllers                   | Списък на контролерите (администратор)           |
| POST   | /api/controllers                   | Създаване на контролер (администратор)           |
| POST   | /api/controllers/claim             | Заявяване с код за сдвояване                     |
| DELETE | /api/controllers/:id               | Изтриване на контролер (администратор)           |
| GET    | /api/users/:userId/controllers     | Контролери на конкретен потребител               |
| POST   | /api/users/:userId/controllers     | Приписване на контролер на потребител            |
| PATCH  | /api/users/:userId/controllers/:id | Преименуване на приписване                       |
| DELETE | /api/users/:userId/controllers     | Премахване на приписване                         |
| GET    | /api/devices                       | Списък на известните устройства                  |
| GET    | /api/latest/:deviceId              | Последно измерване за устройство                 |
| GET    | /api/history/:deviceId             | Исторически данни по брой часове                 |



|        |                   |                                         |
|--------|-------------------|-----------------------------------------|
| GET    | /api/readings     | Странициране и филтриране на измервания |
| GET    | /api/audit        | Записи в одит лога (администратор)      |
| DELETE | /api/audit        | Изчистване на одит лога (администратор) |
| GET    | /api/admin/health | Системно здраве (администратор)         |





#### 4.5 Логика за търсене и филтриране

Маршрутът GET /api/readings поддържа гъвкаво поле-базирано търсене. Потребителят може да комбинира условия за температура, влажност, звук, осветеност, качество на въздуха, устройство и дата, и да сортира и странициране резултатите.

| Синтаксис                    | Пример                      | Описание                                    |
|------------------------------|-----------------------------|---------------------------------------------|
| d:стойност                   | d:esp32                     | Съвпадение по идентификатор на устройство   |
| t:>стойност /<br>t:<стойност | t:>25                       | Температура по-голяма или по-малка от число |
| h:>стойност /<br>h:<стойност | h:<70                       | Влажност по-голяма или по-малка от число    |
| s:мин-макс                   | s:500-800                   | Ниво на звук в диапазон                     |
| lux:>стойност                | lux:>1000                   | Осветеност над стойността                   |
| ts:дата                      | ts:2025-03-15               | Всички измервания за конкретна дата         |
| sort=поле&dir=asc/desc       | sort=temperature_c&dir=desc | Сортиране по поле                           |
| limit=N&offset=M             | limit=50&offset=100         | Странициране на резултатите                 |

#### 4.6 Уеб приложение с React

Уеб клиентът е изграден с React 19 и Vite, стилизиран с Tailwind CSS. Поддържа превод на потребителския интерфейс с отделни речници за английски и български. Компонентите са организирани в страни-ниво контейнери и по-малки повторно използвани секции.

Компонентите на таблото за управление (frontend/src/components/dashboard/) включват: заглавна лента с навигация; лента за статус на свързаност; мрежа от карти за всеки сензор; времеви серии с Recharts за историческите данни; форма за заявяване с код за вдвояване. Административните компоненти обхващат панели за управление на потребители, контролери и приписвания. Отделни компоненти предоставят преглед на одит лого и показатели за системното здраве.



Управлението на автентикацията се осъществява чрез hooks. Access токена се съхранява в паметта на приложението, а refresh токена в localStorage. При изтичане на access токена приложението автоматично заявява нов чрез refresh маршрута, без прекъсване на потребителското изживяване. Таблото се обновява на всеки 30 секунди:

Интернационализацията е реализирана чрез файлово-базирани речници. Всеки ключ в en.ts има съответствие в bg.ts. Превключването между езиците се управлява от React context, достъпен от всеки компонент. Тази организация улеснява добавянето на нови езици и поддържането на консистентен превод при промяна на интерфейса. Recharts времевите серии визуализират историческите данни с интерактивни подсказки (tooltips), показващи точната стойност и времеви печат при задържане на мишката. Потребителят може да избере период за преглед: последния час, 6 часа, 24 часа или 7 дни.

Компонентът SensorGrid показва последните стойности на всеки сензор с цветово кодиране, указващо нивото — зелено за нормални стойности, жълто за гранични и червено за критични. Прагите за кодиране се изчисляват в помощните функции в utils/, базирани на стандартните комфортни стойности за помещения. Например температура между 18 и 26 °C се показва в зелено, стойности под 15 °C или над 30 °C — в червено. По подобен начин относителна влажност между 40 и 60% е нормална, а стойности под 30% или над 70% са критични.

```
// frontend/src/components/Dashboard.tsx
const HISTORY_REFRESH_INTERVAL_MS = 30000;
const LIVE_REFRESH_INTERVAL_MS = 5000;

useEffect(() => {
  if (!selectedDevice) return;
  const interval = setInterval(fetchLatestReading,
    LIVE_REFRESH_INTERVAL_MS);
  return () => clearInterval(interval);
}, [selectedDevice, fetchLatestReading]);

useEffect(() => {
  if (!selectedDevice) return;
  const interval = setInterval(fetchHistory,
    HISTORY_REFRESH_INTERVAL_MS);
  return () => clearInterval(interval);
}, [selectedDevice, fetchHistory]);
```

## 4.7 Мобилно приложение за Андроид

Мобилният клиент е родно Андроид приложение, написано на Kotlin с Gradle Kotlin DSL. Retrofit с OkHttp осигурява HTTP комуникацията. Access токена се добавя автоматично към всяка заявка чрез OkHttp interceptor, а клиентът изпраща и x-



client: mobile. Данните за токена и refresh токена се съхраняват локално и се използват от слоя на хранилище за вход, изход и запазване на сесията.

Приложението реализира всички функционалности на уеб клиента и допълнително предоставя компонент на началния екран (AppWidget), деклариран в AndroidManifest.xml. Компонентът показва последните стойности на всички сензори директно от началния екран, без отваряне на пълното приложение, и се обновява периодично.

## 4.8 Споделени TypeScript типове

Пакетът packages/shared-types/ съдържа TypeScript интерфейси, използвани едновременно от сървъра и уеб клиента. При промяна на API структурата компилаторът открива несъответствия и на двата края. Основните интерфейси са:

Монорепо структурата позволява споделеният пакет да се импортира директно от сървъра и уеб клиента без публикуване в npm регистър. npm workspaces управляват символните връзки между пакетите автоматично. При промяна на интерфейс Reading — например добавяне на ново поле — TypeScript компилаторът незабавно показва всички места в кода, където новото поле не е обработено. Тази статична проверка заменя голяма част от нуждата от ръчно тестване на интеграцията между слоевете.

```
// packages/shared-types/src/models/user.ts + reading.ts
export interface Reading {
  id: string; device_id: string; ts: string;
  temperature_c: string; humidity_pct: string; lux: string;
  sound: number; sound_dbfs?: string | number | null;
  sound_est_spl?: string | number | null;
  air_quality_raw: number | null; air_baseline_pct?: string | number |
  null;
}
export interface AuthUser {
  id: number; username: string; email: string;
  token?: string; refreshToken?: string; role?: 'user' | 'admin';
  is_admin?: number | boolean | string; must_change_password?: boolean |
  number | string;
}
export interface Controller {
  id: number; device_id: string; label?: string | null; pairing_code:
  string | null;
}
export interface AuditLogEntry {
  id: number; actor_id: number | null; actor_email: string | null;
  action: string; entity_type: string; entity_id: string | null;
  metadata: Record<string, unknown> | null;
  ip_address: string | null; user_agent: string | null; created_at:
  string;
}
export interface UserInviteResponse {
  user: AuthUser;
  tempPassword: string;
}
```



#### 4.9 Технологичен стек

| Слой           | Технология                  | Версия    | Защо е избрана                                                               |
|----------------|-----------------------------|-----------|------------------------------------------------------------------------------|
| Устройство     | ESP32 (Xtensa LX6 двуюдрен) | —         | Евтин MCU с вграден Wi-Fi; достатъчно мощен за TCP/IP и обработка на сензори |
| Устройство     | Arduino C++                 | 2.x       | Опростена firmware разработка; богати библиотеки за сензори                  |
| Мрежа          | MQTT / Eclipse Mosquitto    | 2.x       | Лек pub/sub; устройството е независимо от сървъра                            |
| Инфраструктура | Docker Compose              | 4.x       | Повторяемо локално разгъване; изолация на услугите                           |
| База данни     | PostgreSQL                  | 16        | Трайност; релационен модел за телеметрия и бизнес обекти                     |
| Сървър         | Node.js + TypeScript        | 20 / 5.x  | Управлявано от събития вводно-изводно; прт екосистема                        |
| Сървър         | Express.js                  | 4.x       | Лек HTTP; модулни маршрути; ниска сложност                                   |
| Уеб            | React 19 + Vite             | 19 / 5.x  | Компонентна архитектура; hooks; бърз HMR                                     |
| Уеб            | Recharts                    | 2.x       | Графики за времеви серии базирани на D3.js                                   |
| Мобилно        | Андроид / Kotlin            | SDK 26+   | Стандартен стек; безопасност при нулеви стойности; AppWidget                 |
| Мобилно        | Retrofit + OkHttp           | 2.9 / 4.x | Interface-базиран REST клиент; interceptors за JWT                           |
| Тестване       | Vitest + supertest          | 1.x       | Бърза тестова рамка за Node.js/TypeScript                                    |



## 5. СИГУРНОСТ И НАДЕЖДНОСТ

### 5.1 Защита на паролите

Системата използва алгоритъма `scrypt` за защита на паролите. `Scrypt` е изчислително скъпа функция с висока памет — изисква голямо количество оперативна памет при изчислението, което я прави значително по-устойчива на GPU-базирани атаки в сравнение с MD5 или SHA-256. При регистрация системата генерира произволна криптографска добавка, комбинира я с паролата и изчислява хеша. Само хешът и добавката се записват — оригиналната парола никога не се пази.

### 5.2 JWT автентикация

Access токените са подписани JWT с кратък живот. Подписът гарантира, че никой не може да промени полезния товар на токена без знанието на тайния ключ. Сървърът не пази списък с активни access токени — той ги проверява само чрез подписа. Това прави API слоя без постоянно съхранение на сесии (stateless), което улеснява мащабирането.

Refresh токените са дълго живеещи opaque стойности. Сървърът съхранява само SHA-256 хешовете им в таблицата `refresh_tokens`. Ротацията означава, че при всяко обновяване старият токен се отбелязва като използван и се издава нов. Опит за повторна употреба на вече ротиран токен се засича автоматично. Изходът отменя конкретния refresh токен, което прекратява сесията.

Системата поддържа множество едновременни сесии за един потребител — всяко влизане от различно устройство създава отделен refresh токен в таблицата. Изходът отменя само конкретния токен на текущото устройство. Ако администраторът иска да прекрати всички сесии на потребител едновременно, може да постави флаг `revoked_at` на всички редове за съответния `user_id`. Тази гъвкавост е полезна при предполагаем пробив на акаунт.

При повторна употреба на вече ротиран refresh токен системата може да приложи стратегия за засичане на кражба: да отмени всички активни сесии на потребителя като превантивна мярка. Тази по-агресивна реакция е опционална — в текущата реализация системата само отхвърля заявката. Прилагането на автоматична отмяна на всички сесии при засичане на повторна употреба е идентифицирано като подобрение за производствено разгъване.



### 5.3 Ролово разграничение на достъпа

Двете роли прилагат принципа на минималните необходими привилегии. Обикновеният потребител вижда само устройствата, приписани му в таблицата `user_controllers`. При всяка заявка за данни от устройство се изпълнява база данни заявка, проверяваща дали потребителят има право на достъп. Администраторът има достъп до всички системни функции чрез допълнителни маршрути.

### 5.4 Одит лог

Всички чувствителни операции се записват автоматично в таблицата `audit_logs`: вход, смяна на парола, заявяване на устройство, изтриване на потребителски профил, промяна на роля. Всеки запис включва идентификатор на извършителя, вид на действието, засегнатия обект, интернет адреса и заглавието на браузъра. Одит логът е достъпен само за администратори.

### 5.5 Текущи ограничения

Системата е значително по-сигурна от прототипа с обикновен хедър, но все още не е пълноценна производствена платформа. Липсват: ограничение на броя заявки за защита срещу brute-force; многофакторна автентикация; централизирано завъртане на тайния ключ за JWT; HTTPS в конфигурацията за локална разработка. Тези подобрения са идентифицирани като приоритетни за производствено разгъване.

### 5.6 Надеждност

PostgreSQL осигурява трайност на записаните данни. Програмирането на ESP32 включва логика за автоматично повторно свързване при загуба на Wi-Fi или прекъсване на MQTT. Разделянето на модулът за приемане на телеметрия от API сървър означава, че измерванията продължават да се записват дори при временен срив на API. Health endpoint предоставя оперативен статус в реално време.



## 6. ТЕСТВАНЕ И ВАЛИДИРАНЕ

### 6.1 Стратегия за тестване

Проектът следва многослойна стратегия за тестване, обхващаща сървър, уеб клиента и Android приложението. Автоматизираното тестване покрива основните маршрути на API, автентикационния поток и помощните функции. Допълва се от ръчна проверка в реална среда с физически ESP32.

Тестовата пирамида следва принципа: основата са бързите единични тестове, средното ниво са route-level тестовете на API с mock-нат PostgreSQL слой, а върхът е ръчната проверка от край до край. Това прави тестовете бързи, изолирани и повторяеми, макар да не представляват пълни интеграционни тестове с реална база данни.

### 6.2 Тестове на API сървър

Тестовете използват Vitest и supertest с mock на pg слоя. Покриват: регистрация и вход; издаване и проверка на JWT; обновяване и ротация на refresh токени; защитени маршрути (401 без токен, 403 при недостатъчни права); основни операции с данни за измервания и контролери.

```
// backend/src/api.test.ts – примерни тестове
describe('POST /api/auth/login', () => {
  it('върща 401 при неверни данни', async () => {
    const res = await request(app).post('/api/auth/login')
      .send({ email: 'wrong@test.com', password: 'bad' });
    expect(res.status).toBe(401);
  });

  it('върща token и refreshToken при успешен вход', async () => {
    const res = await request(app).post('/api/auth/login')
      .send({ email: testUser.email, password: testUser.rawPassword });
    expect(res.status).toBe(200);
    expect(res.body.token).toBeDefined();
    expect(res.body.refreshToken).toBeDefined();
  });
});

describe('POST /api/auth/refresh', () => {
  it('издава нов token при валиден refreshToken', async () => {
    const res = await request(app).post('/api/auth/refresh')
      .send({ refreshToken: validRefreshToken });
    expect(res.status).toBe(200);
    expect(res.body.token).toBeDefined();
  });
});
```



### **6.3 Тестове на уеб клиента**

Frontend тестовете с Vitest покриват API клиента, помощните функции за форматиране и ролевите проверки, както и отделни dashboard/admin секции. При уеб клиента е реализирана и логика за опресняване на токена при 401 чрез Axios interceptor.

### **6.4 Тестове на Android приложението**

Мобилните тестове с Gradle обхващат слоя на хранилище и помощните класове за визуализация на измерванията. Текущият Android клиент не съдържа автоматичен 401 refresh interceptor, затова тази логика не следва да се описва като реализирана в мобилния слой.

### **6.5 Ръчна проверка**

Системата е проверена в реална среда с физически ESP32, свързан към AM2302, TEMT6000X01, MQ-135 и MAX4466, публикуващ измервания на всеки 5 секунди към локален Mosquitto. Верифицирани са: пълният поток от сензорно измерване до визуализация; вход, обновяване и изход на сесията; заявяване с код за вдвояване; автоматично обновяване на таблото.





## 6.6 Обобщение

Изборът на четири физически сензора с различни принципи на действие илюстрира разнообразието от подходи в IoT хардуера. AM2302 използва цифров протокол с вградена калибрация и предава вече обработени данни. TEMT6000X01 е чисто аналогов елемент — фототок, пропорционален на осветеността. MQ-135 измерва химична реакция — промяна на електрическата съпротива на SnO<sub>2</sub> при контакт с газови молекули. MAX4466 преобразува акустична енергия в електрически сигнал чрез пиезоелектричен ефект в микрофонния капсул. Тази разнородност изисква различни подходи при четене: 1-wire протокол за AM2302, директен АЦП за останалите три. ESP32 обединява всички тези интерфейси в едно компактно устройство, което е ключова причина за неговия избор.

Проектът демонстрира как съвременните инструменти за разработка намаляват несъответствията между слоевете на системата. TypeScript типовете в споделения пакет гарантират, че форматът на JSON съобщенията от сървъра съответства на очакванията на уеб клиента. Версионизираният SQL миграции гарантират, че схемата на базата данни е в известно и контролируемо състояние. Docker Compose гарантира, че инфраструктурните услуги стартират с еднакви параметри на всяка машина. Всеки от тези инструменти решава конкретен проблем на интеграция, произтичащ от сложността на многокомпонентна система.

Системата е проектирана с ясното намерение да бъде разбираема от един програмист. Всяка взета архитектурна решение е документирана с обосновката, която я е породила — защо MQTT пред HTTP, защо PostgreSQL пред документна база данни, защо монорепо пред разделен подход, защо JWT с refresh ротация пред прост токен. Тази прозрачност прави проекта не само функционален прототип, но и образователен ресурс за разбиране на принципите на изграждане на сигурни, многослойни системи за наблюдение на данни от Интернет на нещата.

Разработката на системата потвърждава, че интегрирането на хардуерен, сървърен, уеб и мобилен слой в единна кодова база е постижимо при правилен избор на инструменти. Монорепо подходът с npm workspaces намалява дублирането и подобрява последователността. JWT автентикацията с refresh ротация предоставя практически баланс между сигурност и удобство. Реалните данни от физически сензори — AM2302, TEMT6000X01, MQ-135 и MAX4466 — доказват, че системата работи в действителна среда, не само в симулация. Проектът показва, че съвременният



пълнен стек от технологии позволява на малък екип да изгради цялостна, сигурна и разширяема IoT платформа в рамките на учебна програма.

Компонентът на началния екран за Андроид е конкретен пример за разширяемостта на архитектурата. Той е самостоятелна компонента, която комуникира с API сървър независимо от основното приложение. Потребителят вижда последните стойности на температурата, влажността и качеството на въздуха директно от началния екран на смартфона, без да отваря приложението. Тази функционалност е реализирана чрез стандартния Android AppWidget механизъм и периодично опреснява данните си чрез AlarmManager. Тя демонстрира, че REST API с JWT автентикация може да обслужва различни видове клиенти — пълноценно приложение, уеб таблица и лека компонента на началния екран — с еднакъв интерфейс.

## 7. ЗАКЛЮЧЕНИЕ

### 7.1 Постигнати резултати

Проектирана и реализирана е пълноценна платформа за наблюдение на данни от Интернет на нещата, покриваща целия технологичен стек от хардуерно ниво до потребителски интерфейс. Всички десет функционални изисквания са изпълнени:

- ФИ-01: Регистрация и вход с хеширане на паролата чрез bcrypt и JWT автентикация
- ФИ-02: Двуролова система (потребител / администратор) с проверки при всяка заявка
- ФИ-03: Пълен административен достъп чрез специализиран панел и разширени маршрути
- ФИ-04: ESP32 с AM2302, TEMT6000X01, MQ-135 и MAX4466 събира данни на всеки 5 секунди
- ФИ-05: MQTT публикуване с Eclipse Mosquitto — автоматично без ръчна намеса
- ФИ-06: Трайно съхранение в PostgreSQL с 11 миграции на схемата
- ФИ-07: React уеб табло и Андроид приложение с карти, графики и таблици



- ФИ-08: Автоматично обновяване на всеки 30 секунди без ръчно презареждане
- ФИ-09: Пълна история с поле-базирано търсене, сортиране и странициране
- ФИ-10: Обратна връзка при грешки, изтекъл токен и недостъпни устройства

## 7.2 Оценка на изпълнението

Основната цел — изграждане на пълен конвейер за наблюдение — е постигната и верифицирана в реална среда. JWT автентикацията с refresh token ротация е съществено подобрение спрямо прототипния подход и представлява практически приложим модел за сигурна автентикация. Монорепо структурата, споделените TypeScript типове и версионизирани миграции демонстрират прилагане на добри инженерни практики.

Реалната интеграция на четири физически сензора — AM2302, TEMT6000X01, MQ-135 и MAX4466 — с документираните технически характеристики показва, че системата работи с реален хардуер, а не само с теоретични примери. Всеки сензор е избран и описан въз основа на официалните технически листове на производителите.

AM2302 е избран заради прецизното цифрово измерване с вградена калибрация и единична жица за комуникация. TEMT6000X01 е предпочетен пред по-масово използвания фоторезистор (LDR), тъй като спектралната му чувствителност е адаптирана към човешкото зрение — стойностите му са по-репрезентативни за реалното усещане за осветеност. MQ-135 е избран заради широкия спектър от детектируеми газове, подходящи за наблюдение на качеството на въздуха в затворени помещения. MAX4466 е предпочетен пред обикновен аналогов микрофон, тъй като включва вграден усилвател с контролируемо усиляване, което осигурява стабилен и измерим сигнал при ниска консумация.

## 7.3 Предложения за бъдещо развитие

- Ограничение на броя заявки за защита срещу brute-force атаки
- HTTPS с TLS сертификат при производствено разгъване
- TLS криптиране на MQTT комуникацията между ESP32 и Mosquitto
- Многофакторна автентикация за административни акаунти
- Известявания при прагови стойности на сензорите



ПРОФЕСИОНАЛНА ГИМНАЗИЯ ПО КОМПЮТЪРНИ НАУКИ И  
МАТЕМАТИЧЕСКИ АНАЛИЗИ, „ПРОФ. МИНКО БАЛКАНСКИ“

- Механизъм за засичане на офлайн устройства чрез периодично сигнализиране
- Добавяне на VH1750 I<sup>2</sup>C сензор за прецизни измервания в лукс (вместо сурова АЦП стойност)
- Калибрация на MQ-135 спрямо специфичен газ с криви от datasheet-a
- Пълно автоматизирано тестово покритие от устройство до клиент
- Аномалия детекция за предупреждение при необичайни показания



## 8. ИЗПОЛЗВАНА ЛИТЕРАТУРА

1. Aosong Electronics Co., Ltd. AM2302 Temperature and Humidity Module Product Manual. Guangzhou, 2023.
2. Android Developers. AppWidget Overview. Google LLC, 2024. [онлайн] <https://developer.android.com/develop/ui/views/appwidgets> [прегледан 10.03.2026]
3. Банков, М. Интернет на нещата — концепции и приложения. Техника, София, 2022. ISBN 978-954-03-0124-7
4. Docker Inc. Docker Compose Documentation. 2024. [онлайн] <https://docs.docker.com/compose/> [прегледан 10.03.2026]
5. Eclipse Foundation. Eclipse Mosquitto — An Open Source MQTT Broker. 2023. [онлайн] <https://mosquitto.org/documentation/> [прегледан 10.03.2026]
6. Espressif Systems. ESP32 Technical Reference Manual. Version 5.2. Espressif Systems, 2024.
7. Expressjs.com. Express 4.x API Reference. OpenJS Foundation, 2023. [онлайн] <https://expressjs.com/en/4x/api.html> [прегледан 15.02.2026]
8. IETF RFC 7914. The scrypt Password-Based Key Derivation Function. IETF, 2016.
9. IETF RFC 7519. JSON Web Token (JWT). IETF, 2015. [онлайн] <https://tools.ietf.org/html/rfc7519>
10. JetBrains. Kotlin Programming Language — Official Documentation. 2024. [онлайн] <https://kotlinlang.org/docs/> [прегледан 20.02.2026]
11. Maxim Integrated (Analog Devices). MAX4465–MAX4469 Low-Cost Micropower Microphone Preamplifiers Datasheet. Rev. 2, 2012.
12. MQTT.org. MQTT Version 3.1.1 Protocol Specification. OASIS Standard, 2014. [онлайн] <https://mqtt.org/mqtt-specification/>
13. Node.js Foundation. Node.js v20 Documentation. OpenJS Foundation, 2023. [онлайн] <https://nodejs.org/docs/latest-v20.x/api/> [прегледан 15.02.2026]
14. PostgreSQL Global Development Group. PostgreSQL 16 Documentation. 2023. [онлайн] <https://www.postgresql.org/docs/16/> [прегледан 20.01.2026]



15. React. React v19 — What's New. Meta Open Source, 2024. [онлайн]  
<https://react.dev/blog/2024/12/05/react-19> [прегледан 20.01.2026]
16. Recharts. Recharts — Composable charting library. 2023. [онлайн]  
<https://recharts.org/en-US/> [прегледан 22.01.2026]
17. Square Inc. Retrofit 2 — A type-safe HTTP client for Android. 2023. [онлайн]  
<https://square.github.io/retrofit/> [прегледан 20.02.2026]
18. Tailwind Labs. Tailwind CSS v3 Documentation. 2023. [онлайн]  
<https://tailwindcss.com/docs> [прегледан 25.01.2026]
19. Vite. Vite 5.0 Guide. Evan You, 2023. [онлайн] <https://vitejs.dev/guide/> [прегледан 25.01.2026]
20. Vitest. Vitest — A Vite-native testing framework. 2023. [онлайн] <https://vitest.dev/> [прегледан 25.01.2026]
21. Vishay Semiconductors. TMT6000X01 Ambient Light Sensor Datasheet. Document Number 81579, Rev. 1.9, 2011.
22. Winsen Electronics. MQ-135 Gas Sensor Technical Data. Zhengzhou Winsen Electronics Technology Co., Ltd., Ver. 1.4.



## 9. ПРИЛОЖЕНИЯ

### Приложение А — Технически листове на сензорите

Следната таблица обобщава основните електрически характеристики на четирите сензора, използвани в хардуерния слой на системата:

| Сензор      | Производител          | Измерван параметър                                                                  | Интерфейс          | Захранване |
|-------------|-----------------------|-------------------------------------------------------------------------------------|--------------------|------------|
| AM2302      | Aosong Electronics    | Температура (−40..80°C) и влажност (0..99.9%RH)                                     | 1-wire             | 3.3–5.5 V  |
| TEMT6000X01 | Vishay Semiconductors | Видима светлина (440–800 nm), пик при 570 nm                                        | Аналогов (фототок) | 5 V (Vce)  |
| MQ-135      | Winsen Electronics    | NH <sub>3</sub> , NO <sub>x</sub> , CO <sub>2</sub> , бензен, дим (0–5 V аналогово) | Аналогов (АЦП)     | 5 V        |
| MAX4466     | Maxim Integrated      | Ниво на звука (чрез електретен микрофон)                                            | Аналогов (АЦП)     | 2.4–5.5 V  |

### Приложение Б — Environment Variables

| Променлива     | Описание                       | Пример                    |
|----------------|--------------------------------|---------------------------|
| PGHOST         | Адрес на PostgreSQL сървър     | 127.0.0.1                 |
| PGPORT         | Порт на PostgreSQL             | 5432                      |
| PGDATABASE     | Название на базата данни       | iot                       |
| PGUSER         | PostgreSQL потребителско име   | iotuser                   |
| PGPASSWORD     | PostgreSQL парола              | secret                    |
| MQTT_URL       | Адрес на MQTT посредника       | mqtt://localhost:1883     |
| MQTT_TOPIC     | Тема за телеметрия             | iot/+telemetry            |
| JWT_SECRET     | Таен ключ за подписване на JWT | дълга произволна стойност |
| JWT_EXPIRES_IN | Живот на access токена         | 15m                       |



|                      |                               |                       |
|----------------------|-------------------------------|-----------------------|
| REFRESH_EXPIRES_DAYS | Живот на refresh токена в дни | 30                    |
| PORT                 | HTTP порт на API сървъра      | 3000                  |
| CORS_ORIGINS         | Разрешени CORS адреси         | http://localhost:5173 |
| VITE_API_URL         | Адрес на API за уеб клиента   | http://localhost:3000 |

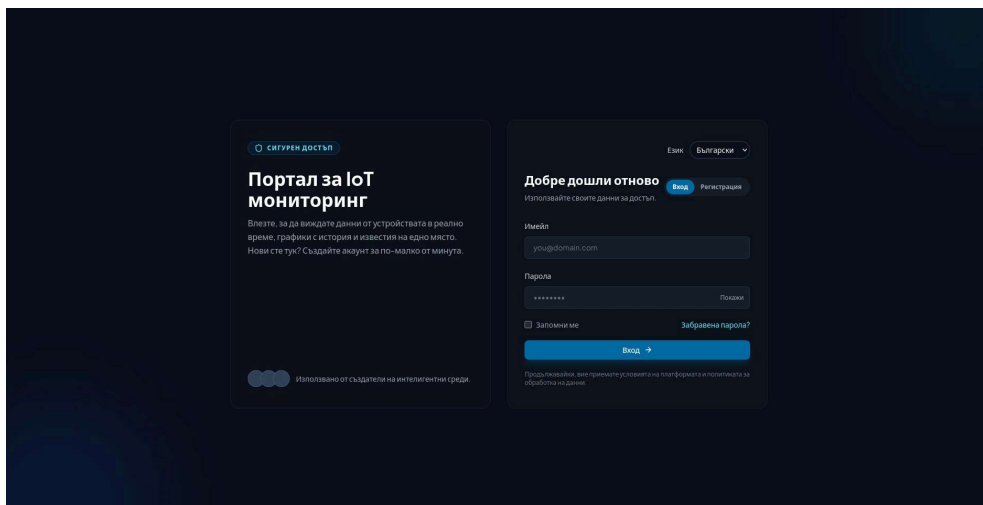
## Приложение В — Качествени характеристики

| Характеристика | Как е осигурена                                                   | Ограничения                                                  |
|----------------|-------------------------------------------------------------------|--------------------------------------------------------------|
| Поддръжаемост  | Монорепо; модулни маршрути; споделени типове; миграции            | —                                                            |
| Надеждност     | Трайност на данните; повторно свързване на ESP32; health endpoint | Без синхрон при офлайн за мобилен клиент                     |
| Сигурност      | scrypt; JWT + refresh ротация; ролеви проверки; одит лог          | Без rate limiting; без MFA; без HTTPS при локална разработка |
| Машабируемост  | Независим модул за приемане на телеметрия; множество клиенти      | Единичен сървърен процес                                     |
| Използваемост  | Карти; графики; мобилен достъп; AppWidget; превод на два езика    | —                                                            |

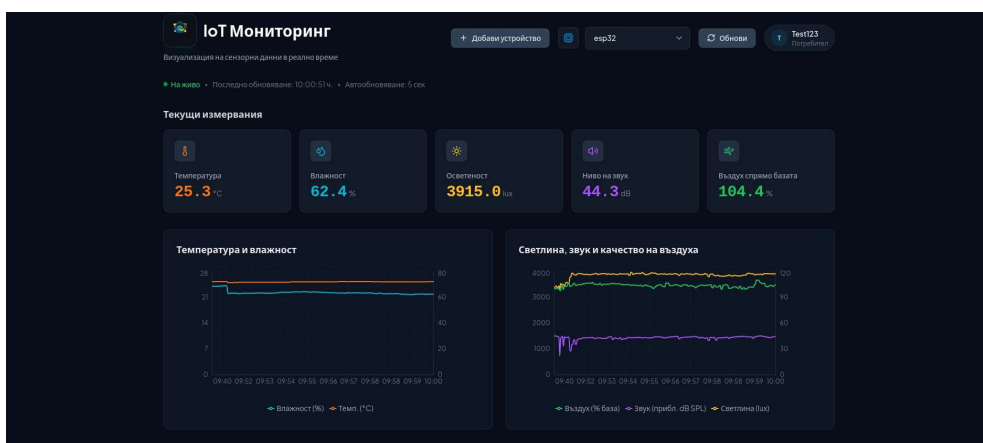
## Приложение Г — Екранни снимки на потребителския интерфейс

Следните екранни снимки илюстрират реализирания потребителски интерфейс на уеб приложението и мобилния клиент.

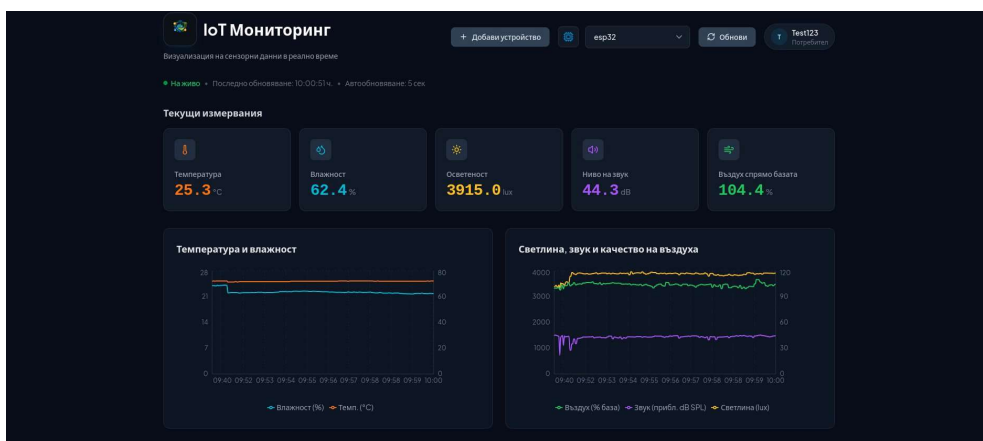




Фигура 1. Форма за вход и регистрация



Фигура 2. Главно табло — мрежа от карти за AM2302, TEMT6000X01, MQ-135 и MAX4466



Фигура 3. Времева серия — история на температурата от AM2302



ПРОФЕСИОНАЛНА ГИМНАЗИЯ ПО КОМПЮТЪРНИ НАУКИ И  
МАТЕМАТИЧЕСКИ АНАЛИЗИ, „ПРОФ. МИНКО БАЛКАНСКИ“

| Сензорни измервания                |            |            |              |                |                      |                 |
|------------------------------------|------------|------------|--------------|----------------|----------------------|-----------------|
| Фърсене на устройства...           |            |            |              |                |                      |                 |
| ВРЕМЕ                              | УСТРОЙСТВО | ТЕМП. (°C) | ВЛАЖНОСТ (%) | СВЕТЛИНА (LUX) | ЗВУК (ПРИБЛ. DB SPL) | ВЪЗДУХ (% БАЗА) |
| 31.03.2026 г., 10:09:01 ч.         | esp32      | 25.4       | 61.3         | 3875           | 36.2                 | 105.8           |
| 31.03.2026 г., 10:09:01 ч.         | esp32      | 25.4       | 61.3         | 3875           | 36.2                 | 105.8           |
| 31.03.2026 г., 10:08:56 ч.         | esp32      | 25.4       | 61.3         | 3883           | 45.6                 | 108.3           |
| 31.03.2026 г., 10:08:56 ч.         | esp32      | 25.4       | 61.3         | 3883           | 45.6                 | 108.3           |
| 31.03.2026 г., 10:08:51 ч.         | esp32      | 25.4       | 61.5         | 3886           | 44.8                 | 108.1           |
| 31.03.2026 г., 10:08:51 ч.         | esp32      | 25.4       | 61.5         | 3886           | 44.8                 | 108.1           |
| 31.03.2026 г., 10:08:46 ч.         | esp32      | 25.4       | 61.5         | 3901           | 44.4                 | 108.0           |
| 31.03.2026 г., 10:08:46 ч.         | esp32      | 25.4       | 61.5         | 3901           | 44.4                 | 108.0           |
| 31.03.2026 г., 10:08:41 ч.         | esp32      | 25.4       | 61.6         | 3870           | 43.9                 | 108.2           |
| 31.03.2026 г., 10:08:41 ч.         | esp32      | 25.4       | 61.6         | 3870           | 43.9                 | 108.2           |
| 31.03.2026 г., 10:08:36 ч.         | esp32      | 25.4       | 61.7         | 3893           | 43.7                 | 108.1           |
| 31.03.2026 г., 10:08:36 ч.         | esp32      | 25.4       | 61.7         | 3893           | 43.7                 | 108.1           |
| 31.03.2026 г., 10:08:31 ч.         | esp32      | 25.4       | 61.6         | 3888           | 43.4                 | 108.1           |
| 31.03.2026 г., 10:08:31 ч.         | esp32      | 25.4       | 61.6         | 3888           | 43.4                 | 108.1           |
| 31.03.2026 г., 10:08:26 ч.         | esp32      | 25.4       | 61.3         | 3905           | 43.2                 | 107.9           |
| Показани 1 до 15 от 459 измервания |            |            |              |                |                      |                 |
| < 1 2 3 ... 31 >                   |            |            |              |                |                      |                 |

Фигура 4. Странициращата таблица с поле-базирано търсене

Заяви контролер

Затвори

Код за свързване

QR код

Код за свързване

5-цифрен код

Етикет на устройство (по избор)

Моят офис сензор

Отказ

Заяви устройство

Фигура 5. Форма за заявяване на устройство с код за сдвояване



## ПРОФЕСИОНАЛНА ГИМНАЗИЯ ПО КОМПЮТЪРНИ НАУКИ И МАТЕМАТИЧЕСКИ АНАЛИЗИ, „ПРОФ. МИНКО БАЛКАНСКИ“

**IoT Мониторинг**

Административно табло

Потребители  
Административен изглед на регистрираните потребители.

Покани потребител

| ПОТРЕБИТЕЛСКО ИМЕ | ИМЕЙЛ                   | РОЛЯ       | ПОКАНЕН | СМЯНА ЗАДЪЛЖИТЕЛНА | СЪЗДАДЕНО                  |
|-------------------|-------------------------|------------|---------|--------------------|----------------------------|
| u_test_1702       | u_test_1702@example.com | Потребител | -       | Не                 | 4.03.2026 г., 12:41:38 ч.  |
| Test123           | pan07nik2@gmail.com     | Потребител | -       | Не                 | 19.01.2026 г., 15:01:38 ч. |
| Niki2k            | pan07nik@gmail.com      | Админ      | -       | Не                 | 19.01.2026 г., 13:17:45 ч. |

Присвояване на контролери  
Присвоявайте контролери (device ids) на потребители.

Потребител: u\_test\_1702 (u\_test\_1702@example.com) | Контролери: Изберете контролер | Присвой

| ID НА КОНТРОЛЕР           | ПРИСВОЕНО | КОД |
|---------------------------|-----------|-----|
| Няма присвоени контролери |           |     |

Контролери  
Създавайте и управлявайте контролери.

ID на устройство: device\_id | Етикет (по избор): Lab Sensor 01 | Добави контролер

| ID НА УСТРОЙСТВО | ЕТИКЕТ | КОД   | СЪЗДАДЕНО                  |
|------------------|--------|-------|----------------------------|
| esp32            | -      | 29077 | 11.03.2026 г., 16:38:50 ч. |

ESP32 IoT Monitoring System • Админ изглед

Фигура 6. Административен панел — управление на потребители и контролери

**IoT Мониторинг**

Одит на активността и историята на промените

Одитен журнал  
Проследявайте потребителски действия и системни промени.

ID на актьор: 123 | Действие: user.login | Тип обект: controller | ID на обект: 42

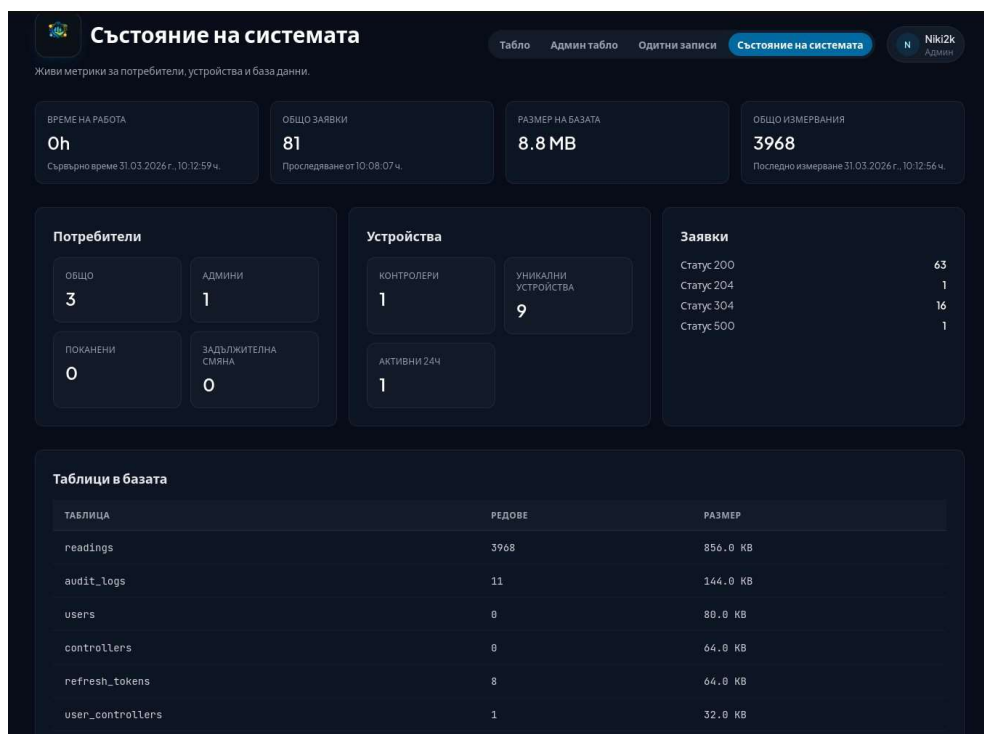
Редове: 20 | Приложи | Изчисти

| ЧАС                        | ИЗВЪРШИТЕЛ          | ДЕЙСТВИЕ            | ОБЕКТ          | ИЗТОЧНИК | IP  | МЕТАДАНИИ                         |
|----------------------------|---------------------|---------------------|----------------|----------|-----|-----------------------------------|
| 31.03.2026 г., 07:11:20 ч. | pan07nik@gmail.com  | user.admin_delete   | user • 4       | web      | ::1 | {"email": "georgijordanov2..."}   |
| 31.03.2026 г., 07:11:08 ч. | pan07nik@gmail.com  | user.login          | user • 1       | web      | ::1 | {"client": "web"}                 |
| 31.03.2026 г., 07:00:51 ч. | pan07nik2@gmail.com | controller.claim    | controller • 7 | web      | ::1 | {"label": null, "client": "w..."} |
| 31.03.2026 г., 07:00:48 ч. | pan07nik2@gmail.com | user.login          | user • 3       | web      | ::1 | {"client": "web"}                 |
| 31.03.2026 г., 07:00:35 ч. | pan07nik@gmail.com  | user.login          | user • 1       | web      | ::1 | {"client": "web"}                 |
| 31.03.2026 г., 07:00:18 ч. | pan07nik2@gmail.com | user.update_profile | user • 3       | web      | ::1 | {"email": "pan07nik2@gmail..."}   |
| 31.03.2026 г., 07:00:09 ч. | pan07nik2@gmail.com | user.login          | user • 3       | web      | ::1 | {"client": "web"}                 |
| 31.03.2026 г., 06:59:50 ч. | pan07nik@gmail.com  | user.login          | user • 1       | web      | ::1 | {"client": "web"}                 |

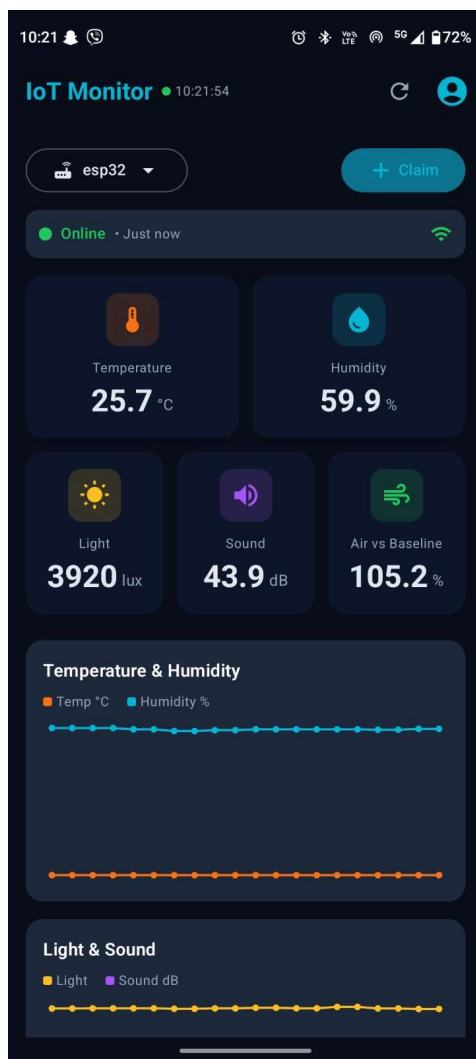
Фигура 7. Одит лог — преглед и филтриране на записи



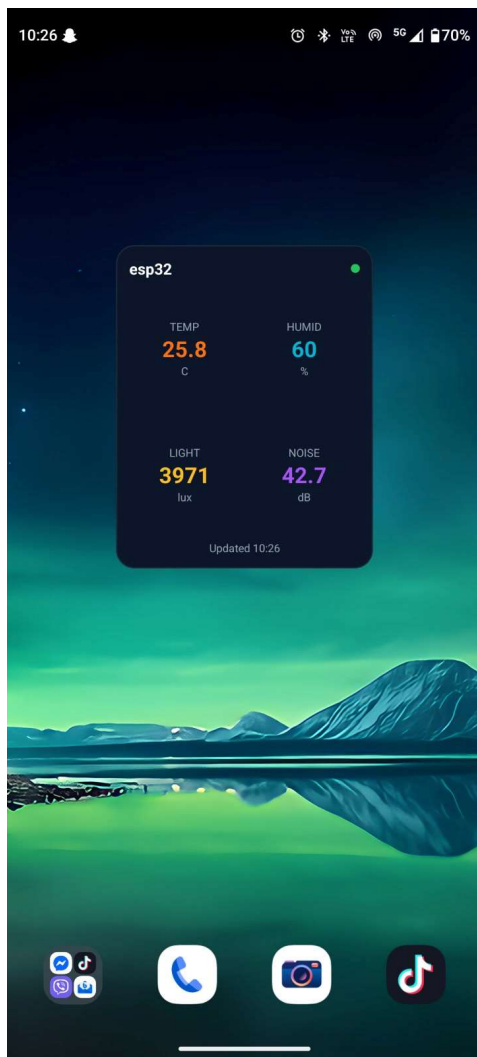
## ПРОФЕСИОНАЛНА ГИМНАЗИЯ ПО КОМПЮТЪРНИ НАУКИ И МАТЕМАТИЧЕСКИ АНАЛИЗИ „ПРОФ. МИНКО БАЛКАНСКИ“



Фигура 8. Показатели за системното здраве



Фигура 9. Мобилно приложение за Андроид — главен екран



Е`Фигура 10. AppWidget на началния екран — последни стойности на сензорите